JAVA OOP

Simple & Complete Study Notes

Object-Oriented Programming in Java

Easy explanations • Short examples • Exam revision • Quick practice

Prepared in simple English with Sinhala-friendly explanations.

Contents

- 1. What is OOP?
- 2. Class and Object
- 3. Constructors
- 4. Encapsulation
- 5. Inheritance
- 6. Polymorphism: Overloading and Overriding
- 7. Abstraction: Abstract Classes and Interfaces
- 8. Access Modifiers
- 9. this and super
- 10. static and final
- 11. Important Keywords and Equality
- 12. Method vs Constructor
- 13. Upcasting and Downcasting
- 14. Association, Aggregation and Composition
- 15. Advantages and Disadvantages
- 16. Quick Revision and MCQs

Tip: Read the short explanation first, then type and run each code example in NetBeans.

1. What is OOP?

OOP (Object-Oriented Programming) is a way of writing programs using classes and objects. It helps organize code into reusable parts.

Example: A Student can have data such as name and age, and behavior such as studying.

The four main OOP principles

- **Encapsulation:** Protect and control access to data.
- **Inheritance:** Reuse properties and methods from another class.
- **Polymorphism:** One method name or reference can work in different ways.
- **Abstraction:** Show essential features and hide implementation details.

2. Class and Object

Class: A blueprint used to create objects. **Object:** An instance created from a class.

```
class Car {
    String color;
}

class Demo {
    public static void main(String[] args) {
        Car c = new Car();
        c.color = "Red";
        System.out.println(c.color);
    }
}
```

Remember: `Car` is the class, `c` is a reference variable, and `new Car()` creates an object.

3. Constructors

A constructor initializes an object when it is created. Its name must match the class name, and it has no return type—not even `void`.

```
class Student {
    String name;

    Student() {                 // no-argument constructor
        name = "Unknown";
    }

    Student(String n) {         // parameterized constructor
        name = n;
    }
}
```

- **Default constructor:** Supplied by the compiler only when you declare no constructor.
- **No-argument constructor:** A constructor with no parameters; it may be written by the programmer.
- **Parameterized constructor:** Accepts values through parameters.
- **Constructor overloading:** Multiple constructors with different parameter lists.

If you write any constructor yourself, Java does not automatically add the default constructor.

4. Encapsulation

Encapsulation combines data and methods in a class and controls direct access to the data. A common pattern is to use private fields with public getters and setters.

```
class Student {
    private int age;

    public void setAge(int age) {
        if (age >= 0) {
            this.age = age;
        }
    }

    public int getAge() {
        return age;
    }
}
```

- `private` limits direct access to the class.
- `getAge()` returns the age; `setAge()` updates it.
- `this.age` refers to the current object's field.

Benefit: Data is easier to protect and validate.

5. Inheritance

Inheritance lets a child class use accessible fields and methods from a parent class. Java classes use `extends`.

```
class Animal {
    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    void bark() {
        System.out.println("Barking");
    }
}

class Demo {
```

```
    public static void main(String[] args) {
        Dog d = new Dog();
        d.eat();
        d.bark();
    }
}
```

Output: `Eating`, then `Barking`.

Common inheritance structures

- **Single:** A → B
- **Multilevel:** A → B → C
- **Hierarchical:** A → B and A → C
- **Multiple:** One class inheriting from multiple classes. Java does not support this directly for classes; interfaces can provide similar multiple-type behavior.
- **Hybrid:** A combination of inheritance structures, possible with classes and interfaces.

Benefits: Code reuse, less duplication, and clearer relationships. Use inheritance when there is a genuine “is-a” relationship.

6. Polymorphism

Polymorphism means the same method name or parent-type reference can represent different behavior.

A. Compile-time polymorphism — Overloading

Methods have the same name but different parameter lists.

```
class Calculator {
    int add(int a, int b) {
        return a + b;
    }

    int add(int a, int b, int c) {
        return a + b + c;
    }
}
```

You can overload by changing the number, types, or order of parameters. Changing only the return type is not enough.

B. Runtime polymorphism — Overriding

A child class provides its own implementation of an inherited instance method.

```
class Animal {
    void sound() {
        System.out.println("Animal sound");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}

class Demo {
    public static void main(String[] args) {
        Animal a = new Dog();
        a.sound();              // Bark
    }
}
```

The reference type is `Animal`, but the object is a `Dog`. At runtime, the overridden method for the actual object runs.

Overloading	Overriding
Different parameter lists	Same method signature (with compatible return type)
Often within one class	Parent-child relationship
Selected using compile-time information	Instance method implementation selected at runtime
Inheritance not required	Inheritance required

7. Abstraction

Abstraction shows what an object can do while hiding details about how it does it. Java supports abstraction through abstract classes and interfaces.

A. Abstract class

```
abstract class Animal {
    abstract void sound();

    void eat() {
        System.out.println("Eating");
    }
}

class Dog extends Animal {
    @Override
    void sound() {
        System.out.println("Bark");
    }
}
```

- You cannot directly create an object of an abstract class.
- An abstract method has no body.
- A concrete child class must implement inherited abstract methods, unless it is also abstract.

B. Interface

```
interface Flyable {
    void fly();
}

class Bird implements Flyable {
    public void fly() {
        System.out.println("Flying");
    }
}
```

- `implements` connects a class to an interface.
- An implementing class must provide public implementations of interface abstract methods, unless the class is abstract.
- Interfaces may also contain default and static methods.

Abstract class	Interface
Extended with extends	Implemented with implements
Can have constructors and instance fields	No constructors or ordinary instance fields
A class can extend only one class	A class can implement multiple interfaces
Useful for shared state and behavior	Useful for defining a common contract

8. Access Modifiers

Access modifiers control where a class member can be accessed.

Modifier	Meaning
public	Accessible wherever the declaring type/member is accessible.
private	Accessible directly only inside the declaring class.
protected	Accessible in the same package and, subject to Java rules, in subclasses in other packages.
default (no modifier)	Accessible within the same package.

```
public int a;
private int b;
protected int c;
int d;                  // package-private (default)
```

9. this and super

this

`this` refers to the current object. It is often used when a field and parameter have the same name.

```
class Student {
    int age;

    Student(int age) {
        this.age = age;
    }
}
```

super

`super` refers to the parent-class part of the object. It can call a parent constructor or an accessible parent method.

```
class Animal {
    Animal() {
        System.out.println("Animal");
    }
}

class Dog extends Animal {
    Dog() {
        super();
        System.out.println("Dog");
    }
}
```

Output: `Animal`, then `Dog`.

- A constructor call using `super(...)` must be the first statement in a constructor.
- If you do not write a constructor invocation, Java inserts a no-argument `super()` where applicable. The parent must have an accessible no-argument constructor.
- Constructors are not inherited or overridden.

10. static and final

static

A static member belongs to the class rather than to each individual object.

```
class Student {
    static String school = "ABC";
}

// Access using:
```

```
System.out.println(Student.school);
```

final

`final` restricts reassignment or inheritance/overriding depending on where it is used.

- **final variable:** Can be assigned only once.
- **final method:** Cannot be overridden.
- **final class:** Cannot be extended.

```
final int MAX = 100;
```

11. Important Keywords and Equality

Keyword / expression	Purpose
new	Creates an object (or array).
extends	Extends a class or interface.
implements	Implements one or more interfaces.
abstract	Declares an abstract class or method.
instanceof	Tests whether an object is compatible with a type.
==	Compares primitive values; for references, checks whether they refer to the same object.
.equals()	Checks logical equality as defined by the class implementation.
@Override	Tells the compiler that a method is intended to override a parent method.
null	A reference value that points to no object.

```
String a = new String("Java");
String b = new String("Java");

System.out.println(a == b);       // false
System.out.println(a.equals(b));  // true
```

The two String objects are different objects, but their text is equal.

12. Method vs Constructor

Method	Constructor
Can have any valid method name	Name must match the class
Has a return type (or void)	Has no return type
Called to perform behavior	Runs during object creation
Can be inherited/overridden when rules allow	Not inherited and cannot be overridden

If you write `void Student()`, that is a method, not a constructor.

13. Upcasting and Downcasting

Upcasting

Treating a child object as a parent type. It is usually automatic.

```
Animal a = new Dog();
```

Downcasting

Converting a parent-type reference back to a child type. Check the actual object first.

```
if (a instanceof Dog) {
    Dog d = (Dog) a;
}
```

Casting to an incompatible object type can cause `ClassCastException`. Modern Java also supports pattern matching for instanceof, but the classic cast above is useful for learning basics.

14. Association, Aggregation and Composition

- **Association:** A general relationship between classes, e.g., Teacher and Student.
- **Aggregation:** A weaker “has-a” relationship; the parts can exist independently, e.g., a Department and Teachers.
- **Composition:** A stronger “has-a” relationship in which the part's lifecycle is usually tied to the whole, e.g., a House and Rooms in a simplified model.

These are design relationships; the exact lifecycle depends on how the program is designed.

15. Advantages and Disadvantages

Advantages

- Code can be reused.
- Programs can be organized into clear components.
- Encapsulation helps protect data.
- Maintenance and updates can be easier.
- Polymorphism can make code more flexible.

Disadvantages

- OOP concepts may take time to learn.
- Small programs may need more code than a simple procedural solution.
- Objects and abstractions can add memory or runtime overhead.

16. Quick Revision

Concept	One-line memory tip
Class	Blueprint for objects.
Object	An instance of a class.
Constructor	Initializes an object; no return type.
Encapsulation	Protect data with controlled access.
Inheritance	Reuse parent behavior with extends.
Overloading	Same method name, different parameter lists.
Overriding	Child provides a new implementation of an inherited method.
Abstraction	Expose essentials, hide implementation details.
Interface	A contract a class can implement.
this	Current object.
super	Parent-class members or constructor.
static	Belongs to the class.
final	Restricts reassignment, overriding, or extension.

Quick MCQ Practice

1. Which keyword creates an object?

- A. extends
- B. new
- C. super
- D. static

Answer: B. new

2. Which keyword is used for class inheritance?

- A. implements
- B. this
- C. extends
- D. final

Answer: C. extends

3. What is method overloading?

- A. Same name, different parameter lists
- B. Same parameters, only different return type
- C. Calling a parent constructor
- D. Making a field private

Answer: A. Same name, different parameter lists

4. Runtime polymorphism is commonly achieved through:

- A. Constructor overloading
- B. Method overriding
- C. Private variables
- D. Static fields

Answer: B. Method overriding

5. Can an abstract class be instantiated directly?

- A. Yes
- B. No

Answer: B. No

Study tip: Learn the definition, understand the example, and practise writing the code without looking at the note.